

UNIVERSITÉ — DÉPARTEMENT INFORMATIQUE DE GESTION

Projet Tutoré — Fin de Cycle de Licence (L3)

SYSTÈME DE GESTION DES CLIENTS REGIDESO

Site web — Code source commenté

Réalisé par :

NGANDU NGANDU JEAN

Étudiant en L3 Informatique de Gestion

Cas de la REGIDESO — Régie de Distribution de l'Eau

Année académique 2025 — 2026

TABLE DES MATIÈRES

1. Architecture du projet	3
2. Base de données — Schéma des tables	4
2.1 Table clients	4
2.2 Table consommations	4
2.3 Table factures	5
2.4 Table réclamations	5
3. API Backend — Routes Express	6
3.1 Routes Clients (CRUD)	6
3.2 Routes Consommations	9
3.3 Routes Réclamations	11
3.4 Routes Tableau de Bord	13
4. Frontend React — Application principale	15
4.1 Point d'entrée App.tsx (routage)	15
4.2 Page d'accueil (home.tsx)	16
4.3 Tableau de bord (dashboard.tsx)	18

1. ARCHITECTURE DU PROJET

Le projet est structuré en monorepo avec trois couches principales :

```
Ø=ÜÄ workspace/
  Ø=ÜÄ lib/
    Ø=ÜÄ db/src/schema/           !• Définition des tables PostgreSQL (Drizzle ORM)
    Ø=ÜÄ api-spec/               !• Contrat OpenAPI (description des endpoints)
    Ø=ÜÄ api-client-react/       !• Hooks React Query générés automatiquement
    Ø=ÜÄ api-zod/                !• Schémas de validation Zod générés
  Ø=ÜÄ artifacts/
    Ø=ÜÄ api-server/src/routes/  !• Serveur Express : logique métier backend
    Ø=ÜÄ regideso/src/pages/     !• Interface React : pages utilisateur
    Ø=ÜÄ regideso/src/components/!• Composants réutilisables (sidebar, layout...)
```

Technologies utilisées :

- **Node.js 24 + TypeScript** — Langage du serveur et du client (typage fort)
- **Express 5** — Framework HTTP pour le serveur backend
- **PostgreSQL + Drizzle ORM** — Base de données relationnelle + ORM TypeScript
- **React + Vite** — Bibliothèque UI + outil de build ultra-rapide
- **TanStack Query (React Query)** — Gestion du cache et des requêtes API côté client
- **Zod** — Validation des données (schémas) côté serveur et client
- **Recharts** — Bibliothèque de graphiques pour le tableau de bord
- **Wouter** — Routeur léger pour React (navigation entre pages)
- **shadcn/ui + Tailwind CSS** — Composants UI préfabriqués + styles CSS utilitaires

2. BASE DE DONNÉES — SCHÉMA DES TABLES

Chaque fichier dans `lib/db/src/schema/` définit une table PostgreSQL avec Drizzle ORM. Drizzle est un ORM TypeScript qui permet d'écrire les requêtes SQL de façon sécurisée et typée.

2.1 `lib/db/src/schema/clients.ts` — Table des abonnés




```

1  import { pgTable, text, serial, timestamp, pgEnum } from "drizzle-orm/pg-core";
   '▫ Importation des types de colonnes PostgreSQL depuis Drizzle ORM
2  import { createInsertSchema } from "drizzle-zod";
   '▫ createInsertSchema génère automatiquement un schéma Zod depuis la table
3  import { z } from "zod/v4";
   '▫ Zod : bibliothèque de validation des données
4
5  export const statutClientEnum = pgEnum("statut_client",
   '▫ Création d'un type ENUM PostgreSQL pour les statuts possibles
6    ["actif", "inactif", "suspendu"]);
   '▫ 3 valeurs autorisées : actif, inactif, suspendu
7
8  export const clientsTable = pgTable("clients", {
   '▫ Définition de la table SQL "clients"
9    id: serial("id").primaryKey(),
   '▫ Colonne id : entier auto-incrémenté, clé primaire
10   numeroAbonnement: text("numero_abonnement").notNull().unique(),
   '▫ N° unique par abonné, obligatoire, format RGD-AAAA-NNNNN
11   nom: text("nom").notNull(),
   '▫ Nom de famille de l'abonné, obligatoire
12   prenom: text("prenom").notNull(),
   '▫ Prénom, obligatoire
13   adresse: text("adresse").notNull(),
   '▫ Adresse physique complète, obligatoire
14   telephone: text("telephone").notNull(),
   '▫ Numéro de téléphone, obligatoire
15   email: text("email"),
   '▫ Email facultatif (nullable par défaut car pas de .notNull())
16   statut: statutClientEnum("statut").notNull().default("actif"),
   '▫ Statut ENUM, défaut "actif" pour tout nouveau client
17   quartier: text("quartier"),
   '▫ Quartier de résidence, facultatif
18   commune: text("commune"),
   '▫ Commune de résidence, facultatif
19   dateInscription: timestamp("date_inscription").notNull().defaultNow(),
   '▫ Date d'inscription, remplie automatiquement à la création
20 });
21
22 export const insertClientSchema = createInsertSchema(clientsTable).omit({
   '▫ Schéma de validation pour l'insertion : on exclut les champs auto-générés
23   id: true, dateInscription: true, numeroAbonnement: true,
   '▫ id, dateInscription et numeroAbonnement sont gérés par le système
24 });
25 export type InsertClient = z.infer<typeof insertClientSchema>;
   '▫ Type TypeScript déduit automatiquement du schéma Zod
26 export type Client = typeof clientsTable.$inferSelect;
   '▫ Type TypeScript d'un client complet lu depuis la base de données

```

2.2 lib/db/src/schema/consommations.ts — Relevés de compteurs


```
1 import { pgTable, serial, integer, numeric, timestamp } from "drizzle-orm/pg-core";  
  '▫ Importation : numeric pour les m³ (décimaux), integer pour mois/année  
2 import { createInsertSchema } from "drizzle-zod";  
  '▫ Génération automatique du schéma de validation Zod  
3 import { clientsTable } from "../clients";  
  '▫ Import de la table clients pour définir la clé étrangère  
4  
5 export const consommationsTable = pgTable("consommations", {  
  '▫ Définition de la table SQL "consommations"  
6   id: serial("id").primaryKey(),  
  '▫ Identifiant unique auto-incrémenté  
7   clientId: integer("client_id").notNull()  
  '▫ Clé étrangère : référence vers l'abonné  
8   .references(() => clientsTable.id, { onDelete: "cascade" })),  
  '▫ Si l'abonné est supprimé, ses consommations le sont aussi (cascade)  
9   mois: integer("mois").notNull(),  
  '▫ Mois du relevé (1 = janvier, 12 = décembre)  
10  annee: integer("annee").notNull(),  
  '▫ Année du relevé (ex: 2025)  
11  m3Consommes: numeric("m3_consommes", { precision:10, scale:2 }).notNull(),  
  '▫ Volume consommé en m³ avec 2 décimales (ex: 12.50)  
12  dateReleve: timestamp("date_releve").notNull(),  
  '▫ Date exacte à laquelle le relevé a été effectué  
13 });
```


2.3 lib/db/src/schema/factures.ts — Factures de paiement

```
1 import { pgTable, serial, integer, numeric, timestamp, pgEnum }  
  '▫ pgEnum pour les statuts de paiement'  
2   from "drizzle-orm/pg-core";  
3 import { clientsTable } from "../clients";  
  '▫ Référence vers la table clients'  
4 import { consommationsTable } from "../consommations";  
  '▫ Référence vers la table consommations (lien facture !" relevé)'  
5  
6 export const statutPaieementEnum = pgEnum("statut_paiement",  
  '▫ ENUM pour le statut de chaque facture'  
7   ["payee", "impayee", "en_attente"]);  
  '▫ 3 états : payée, impayée, ou en attente de confirmation'  
8  
9 export const facturesTable = pgTable("factures", {  
  '▫ Définition de la table SQL "factures"  
10   id: serial("id").primaryKey(),  
  '▫ Identifiant unique'  
11   clientId: integer("client_id").notNull(),  
  '▫ Quel abonné est facturé'  
12   .references(() => clientsTable.id, { onDelete: "cascade" })),  
  '▫ Suppression en cascade si l'abonné est supprimé'  
13   consommationId: integer("consommation_id")  
  '▫ Optionnel : lien vers le relevé de consommation'  
14   .references(() => consommationsTable.id, { onDelete: "set null" })),  
  '▫ set null : si le relevé est supprimé, la facture reste mais sans lien'  
15   montant: numeric("montant", { precision:12, scale:2 }).notNull(),  
  '▫ Montant en Francs Congolais (FC), 2 décimales'  
16   statutPaieement: statutPaieementEnum("statut_paiement").notNull().default("en_attente"),  
  '▫ Par défaut "en_attente" à la création d'une facture'  
17   dateEmission: timestamp("date_emission").notNull().defaultNow(),  
  '▫ Date de génération de la facture, automatique'  
18   dateEcheance: timestamp("date_echeance"),  
  '▫ Date limite de paiement (facultatif)'  
19   datePaieement: timestamp("date_paiement"),  
  '▫ Date réelle du paiement, remplie quand la facture est marquée "payée"  
20   });
```

2.4 lib/db/src/schema/reclamations.ts — Réclamations des abonnés


```
1  export const typeReclamationEnum = pgEnum("type_reclamation",
    '▫ Types de réclamations possibles
2  ["fuite", "facturation", "coupure", "qualite", "autre"]);
    '▫ Fuite d'eau, erreur de facturation, coupure, qualité ou autre
3  export const statutReclamationEnum = pgEnum("statut_reclamation",
    '▫ Cycle de vie d'une réclamation
4  ["ouverte", "en_cours", "resolue", "fermee"]);
    '▫ Ouverte ! En cours ! Résolue ! Fermée
5
6  export const reclamationsTable = pgTable("reclamations", {
    '▫ Table SQL "reclamations"
7  id: serial("id").primaryKey(),
    '▫ Identifiant unique
8  clientId: integer("client_id").notNull()
    '▫ Abonné qui a soumis la réclamation
9  .references(() => clientsTable.id, { onDelete: "cascade" }),
    '▫ Suppression en cascade
10 type: typeReclamationEnum("type").notNull(),
    '▫ Catégorie de la réclamation
11 description: text("description").notNull(),
    '▫ Texte libre décrivant le problème
12 statut: statutReclamationEnum("statut").notNull().default("ouverte"),
    '▫ Toute réclamation commence avec le statut "ouverte"
13 dateCreation: timestamp("date_creation").notNull().defaultNow(),
    '▫ Horodatage automatique de la soumission
14 dateResolution: timestamp("date_resolution"),
    '▫ Rempli quand la réclamation est résolue ou fermée
15 });
```


3. API BACKEND — ROUTES EXPRESS

Le backend est un serveur Express qui expose des endpoints REST. Chaque requête HTTP reçue est validée avec Zod, traitée avec Drizzle ORM, et retourne du JSON.

Méthodes HTTP utilisées :

GET !' Lire des données (liste ou détail) | POST !' Créer un enregistrement
PATCH !' Modifier partiellement | DELETE !' Supprimer un enregistrement

3.1 artifacts/api-server/src/routes/clients.ts — CRUD Abonnés

```
1 import { Router } from "express";  
  '▫ Router Express : permet de grouper les routes d'un même module  
2 import { eq, ilike, or, and } from "drizzle-orm";  
  '▫ eq=égalité, ilike=recherche insensible à la casse, or/and=combinaisons  
3 import { db, clientsTable } from "@workspace/db";  
  '▫ db = connexion à la base; clientsTable = définition de la table  
4 import { ListClientsQueryParams, CreateClientBody, ... }  
  '▫ Schémas Zod générés automatiquement pour valider les entrées  
5   from "@workspace/api-zod";  
6  
7 const router = Router();  
  '▫ Création d'une instance de routeur Express  
8 let abonnementCounter = 0;  
  '▫ Compteur en mémoire pour générer les numéros d'abonnement  
9  
10 function generateNumeroAbonnement(): string {  
11   abonnementCounter++;  
12   const year = new Date().getFullYear();  
13   return `RGD-${year}-${String(abonnementCounter).padStart(5, "0")}`;  
14 }  
  '▫ Résultat : RGD-2026-00001, RGD-2026-00002, etc.
```



```

16 router.get("/clients", async (req, res) => {
    '▫ Endpoint GET /api/clients — retourne la liste de tous les abonnés
17     const parsed = ListClientsQueryParams.safeParse(req.query);
    '▫ Validation des paramètres de l'URL (?search=...&statut=...)
18     if (!parsed.success) {
        '▫ Si la validation échoue
19         return res.status(400).json({ error: "Paramètres invalides" });
        '▫ On retourne HTTP 400 avec un message d'erreur en JSON
20     }
21     const { search, statut } = parsed.data;
    '▫ Extraction des paramètres validés
22     const conditions = [];
    '▫ Tableau de conditions SQL (filtres dynamiques)
23     if (statut) conditions.push(eq(clientsTable.statut, statut));
    '▫ Si un statut est fourni, on filtre par ce statut
24     if (search) conditions.push(or(
        '▫ Si une recherche est fournie, on cherche dans plusieurs colonnes
25         ilike(clientsTable.nom, `%${search}%`),
        '▫ ilike = LIKE insensible à la casse, % = n'importe quels caractères
26         ilike(clientsTable.prenom, `%${search}%`),
        '▫ Recherche aussi dans le prénom
27         ilike(clientsTable.telephone, `%${search}%`),
        '▫ Et dans le téléphone
28         ilike(clientsTable.numeroAbonnement, `%${search}%`)
        '▫ Et dans le numéro d'abonnement
29     ));
30     const clients = conditions.length > 0
        '▫ Si des conditions existent, on les applique, sinon on récupère tout
31     ? await db.select().from(clientsTable).where(and(...conditions))
        '▫ Requête SQL: SELECT * FROM clients WHERE (conditions)
32     : await db.select().from(clientsTable);
    '▫ Sans filtre : SELECT * FROM clients
33     return res.json(clients.map(c => ({
        '▫ On transforme le résultat avant de l'envoyer au client
34         ...c,
        '▫ On copie tous les champs du client
35         dateInscription: c.dateInscription.toISOString(),
        '▫ On convertit la date en chaîne ISO 8601 (format standard JSON)
36     })));
37 });

```



```
39 router.post("/clients", async (req, res) => {
    '▫ Endpoint POST /api/clients — crée un nouvel abonné
40     const parsed = CreateClientBody.safeParse(req.body);
    '▫ Validation du corps de la requête (données envoyées par le formulaire)
41     if (!parsed.success) return res.status(400).json({...});
    '▫ Erreur si les données sont invalides
42     const existing = await db.select().from(clientsTable).limit(1);
    '▫ On vérifie combien de clients existent déjà pour le compteur
43     abonnementCounter = existing.length;
    '▫ Initialisation du compteur selon le nombre de clients
44     const client = await db.insert(clientsTable).values({
    '▫ INSERT INTO clients (...) VALUES (...)
45         ...parsed.data,
    '▫ Insertion de toutes les données validées
46         statut: parsed.data.statut ?? "actif",
    '▫ Statut par défaut "actif" si non fourni
47         numeroAbonnement: generateNumeroAbonnement(),
    '▫ Génération automatique du numéro d'abonnement
48     }).returning();
    '▫ .returning() retourne l'enregistrement inséré complet
49     return res.status(201).json({ ...client[0], ... });
    '▫ HTTP 201 Created avec le client créé
50 });
```

```
52 router.get("/clients/:id", async (req, res) => {
    '▫ Endpoint GET /api/clients/5 — détail d'un seul abonné par son ID
53     const parsed = GetClientParams.safeParse(req.params);
    '▫ Validation de l'ID dans l'URL (doit être un entier)
54     const client = await db.select().from(clientsTable)
    '▫ SELECT * FROM clients WHERE id = ?
55         .where(eq(clientsTable.id, parsed.data.id)).limit(1);
    '▫ On limite à 1 résultat (id est unique)
56     if (client.length === 0) return res.status(404).json({...});
    '▫ Si aucun résultat, HTTP 404 Not Found
57     return res.json({ ...client[0], dateInscription: ... });
    '▫ HTTP 200 avec les données du client
58 });
```



```
60 router.patch("/clients/:id", async (req, res) => {  
    '▫ Endpoint PATCH /api/clients/5 — modification partielle d'un abonné'  
61     const paramsParsed = UpdateClientParams.safeParse(req.params);  
    '▫ Validation de l'ID dans l'URL'  
62     const bodyParsed = UpdateClientBody.safeParse(req.body);  
    '▫ Validation des champs à modifier'  
63     const updated = await db.update(clientsTable)  
    '▫ UPDATE clients SET ... WHERE id = ?'  
64     .set({ ...bodyParsed.data })  
    '▫ Mise à jour des champs fournis uniquement'  
65     .where(eq(clientsTable.id, paramsParsed.data.id))  
    '▫ Condition : seulement le client avec cet ID'  
66     .returning();  
    '▫ Retourne l'enregistrement mis à jour'  
67 });
```

```
69 router.delete("/clients/:id", async (req, res) => {  
    '▫ Endpoint DELETE /api/clients/5 — suppression d'un abonné'  
70     await db.delete(clientsTable).where(eq(clientsTable.id, parsed.data.id));  
    '▫ DELETE FROM clients WHERE id = ? (les données liées supprimées par cascade)'  
71     return res.status(204).send();  
    '▫ HTTP 204 No Content : succès sans corps de réponse'  
72 });  
73  
74 export default router;  
    '▫ Export du routeur pour l'inclure dans le routeur principal'
```


3.2 routes/consommations.ts — Relevés de consommation

```
1  router.get("/consommations", async (req, res) => {
2    ' GET /api/consommations — liste tous les relevés (avec filtres mois/année)
    const { mois, annee } = parsed.data;
3    ' Paramètres optionnels de l'URL pour filtrer
    const conditions = [];
4    ' Tableau dynamique de conditions SQL
    if (mois !== undefined) conditions.push(eq(consommationsTable.mois, mois));
5    ' Filtre par mois si fourni
    if (annee !== undefined) conditions.push(eq(consommationsTable.annee, annee));
6    ' Filtre par année si fournie
    const rows = await db.select({
7      ' SELECT avec jointure : on récupère aussi les infos du client
      id: consommationsTable.id,
8      ' ID du relevé
      clientNom: clientsTable.nom,
9      ' Nom du client lié (JOIN)
      numeroAbonnement: clientsTable.numeroAbonnement,
10     ' N° d'abonnement du client (JOIN)
      m3Consommes: consommationsTable.m3Consommes,
11     ' Volume consommé en m³
    }).from(consommationsTable)
12     ' Table principale : consommations
    .leftJoin(clientsTable, eq(consommationsTable.clientId, clientsTable.id))
13     ' LEFT JOIN clients ON consommations.client_id = clients.id
    .where(and(...conditions));
14     ' Application des filtres
    return res.json(rows.map(r => ({
15       ' Transformation : conversion des types avant envoi
      ...r, m3Consommes: Number(r.m3Consommes),
16       ' numeric !' number JavaScript (Drizzle retourne des strings pour numeric)
      dateReleve: r.dateReleve.toISOString(),
17       ' Date !' chaîne ISO pour le JSON
    })));
18 });
```



```

20 router.post("/consommations", async (req, res) => {
    '▫ POST /api/consommations — enregistre un nouveau relevé
21   const { clientId, mois, annee, m3Consommes, dateReleve } = parsed.data;
    '▫ Extraction des champs validés
22   const inserted = await db.insert(consommationsTable).values({
    '▫ INSERT INTO consommations (...) VALUES (...)
23     clientId, mois, annee,
    '▫ Références et période
24     m3Consommes: String(m3Consommes),
    '▫ Conversion en string : PostgreSQL numeric attend une chaîne
25     dateReleve: new Date(dateReleve),
    '▫ Conversion string à objet Date JavaScript
26   }).returning();
    '▫ Retourne l'enregistrement créé
27   const client = await db.select().from(clientsTable)
    '▫ Requête supplémentaire pour récupérer les infos du client
28     .where(eq(clientsTable.id, row.clientId)).limit(1);
    '▫ SELECT client lié pour enrichir la réponse
29   return res.status(201).json({ ...row, clientNom: client[0]?.nom });
    '▫ HTTP 201 avec le relevé créé enrichi des infos client
30 });

```

```

32 router.get("/clients/:id/consommations", async (req, res) => {
    '▫ GET /api/clients/3/consommations — relevés d'un abonné spécifique
33   const rows = await db.select({...})
    '▫ Même requête JOIN que ci-dessus
34     .from(consommationsTable)
    '▫ Table principale
35     .leftJoin(clientsTable, eq(consommationsTable.clientId, clientsTable.id))
    '▫ Jointure pour avoir le nom et numéro d'abonnement
36     .where(eq(consommationsTable.clientId, parsed.data.id));
    '▫ Filtre : seulement les relevés de CET abonné
37   return res.json(rows.map(r => ({ ...r, m3Consommes: Number(r.m3Consommes) })));
    '▫ Conversion et envoi
38 });

```


3.3 routes/reclamations.ts — Gestion des réclamations

```
1 function mapReclamation(r) {  
  '▫ Fonction utilitaire : transforme une réclamation pour l'envoi JSON'  
2   return {  
3     ...r,  
    '▫ On copie tous les champs'  
4     dateCreation: r.dateCreation.toISOString(),  
    '▫ Date objet ! chaîne ISO 8601'  
5     dateResolution: r.dateResolution?.toISOString() ?? null,  
    '▫ Opérateur ?. : si null/undefined ! on retourne null'  
6   };  
7 }
```

```
9 router.get("/reclamations", async (req, res) => {  
  '▫ GET /api/reclamations — liste toutes les réclamations'  
10   const baseQuery = db.select({...})  
  '▫ Construction de la requête de base avec jointure clients'  
11   .from(reclamationsTable)  
  '▫ Table principale'  
12   .leftJoin(clientsTable, eq(reclamationsTable.clientId, clientsTable.id));  
  '▫ JOIN pour avoir le nom et numéro d'abonnement du client'  
13   const rows = parsed.data.statut  
  '▫ Si un filtre de statut est fourni dans l'URL'  
14   ? await baseQuery.where(eq(reclamationsTable.statut, statut))  
  '▫ On filtre par statut (ouverte, en_cours, etc.)'  
15   : await baseQuery;  
  '▫ Sinon on retourne toutes les réclamations'  
16   return res.json(rows.map(mapReclamation));  
  '▫ Application de la transformation et envoi'  
17 });
```

```
19 router.post("/reclamations", async (req, res) => {  
  '▫ POST /api/reclamations — soumet une nouvelle réclamation'  
20   const inserted = await db.insert(reclamationsTable).values({  
    '▫ Insertion en base de données'  
21     clientId: parsed.data.clientId,  
    '▫ Qui soumet la réclamation'  
22     type: parsed.data.type,  
    '▫ Catégorie (fuite, facturation, coupure...)'  
23     description: parsed.data.description,  
    '▫ Description textuelle du problème'  
24     statut: "ouverte",  
    '▫ Toute nouvelle réclamation commence avec le statut "ouverte"'  
25   }).returning();  
26   return res.status(201).json(mapReclamation({ ...row, clientNom, ... }));  
  '▫ HTTP 201 avec la réclamation créée enrichie'  
27 });
```



```
29 router.patch("/reclamations/:id", async (req, res) => {  
    '▢ PATCH /api/reclamations/3 — met à jour le statut d'une réclamation  
30     const updateData: Record<string, unknown> = {};  
    '▢ Objet vide pour les champs à modifier  
31     if (bodyParsed.data.statut !== undefined)  
        '▢ On n'ajoute que les champs fournis dans la requête  
32         updateData.statut = bodyParsed.data.statut;  
        '▢ Nouveau statut (en_cours, resolue, fermee)  
33     if (bodyParsed.data.dateResolution !== undefined)  
        '▢ Si une date de résolution est fournie  
34         updateData.dateResolution = new Date(bodyParsed.data.dateResolution);  
        '▢ Conversion en Date JavaScript  
35     const updated = await db.update(reclamationsTable)  
        '▢ UPDATE reclamations SET ... WHERE id = ?  
36         .set(updateData).where(eq(reclamationsTable.id, id)).returning();  
        '▢ Mise à jour et récupération du résultat  
37     if (updated.length === 0) return res.status(404).json({...});  
        '▢ Si aucun enregistrement modifié : ID inexistant  
38 });
```


3.4 routes/dashboard.ts — Statistiques du tableau de bord

```
1 import { eq, count, sum, sql } from "drizzle-orm";
  '▫ count() et sum() sont des fonctions d'agrégation SQL
2 import { db, clientsTable, facturesTable,
  '▫ Import de toutes les tables nécessaires
3   reclamationsTable, consommationsTable } from "@workspace/db";

5 router.get("/dashboard/stats", async (req, res) => {
  '▫ GET /api/dashboard/stats — toutes les statistiques en un seul appel
6   const now = new Date();
  '▫ Date et heure actuelles
7   const currentMonth = now.getMonth() + 1;
  '▫ getMonth() renvoie 0-11, on ajoute 1 pour avoir 1-12
8   const currentYear = now.getFullYear();
  '▫ Année courante (ex: 2026)
9
10  const [totalClients] = await db.select({ count: count() })
  '▫ SELECT COUNT(*) FROM clients — nombre total d'abonnés
11    .from(clientsTable);
12  const [clientsActifs] = await db.select({ count: count() })
  '▫ SELECT COUNT(*) FROM clients WHERE statut = 'actif'
13    .from(clientsTable).where(eq(clientsTable.statut, "actif"));
14  const [facturesImpayees] = await db.select({ count: count() })
  '▫ Nombre de factures avec statut "impayee"
15    .from(facturesTable).where(eq(facturesTable.statutPaiement, "impayee"));
16  const [montantImpaye] = await db.select({ total: sum(facturesTable.montant) })
  '▫ SELECT SUM(montant) FROM factures WHERE statut_paiement = 'impayee'
17    .from(facturesTable).where(eq(facturesTable.statutPaiement, "impayee"));
  '▫ Somme totale des montants impayés
18  const [totalConsommationMois] = await db.select({ total: sum(...) })
  '▫ Consommation totale du mois en cours
19    .from(consommationsTable)
20    .where(sql`mois = ${currentMonth} AND annee = ${currentYear}`);
  '▫ Filtre SQL brut pour les deux conditions mois ET année
21  return res.json({
  '▫ On retourne un objet JSON avec toutes les stats
22    totalClients: totalClients.count,
  '▫ Nombre total d'abonnés
23    montantImpaye: Number(montantImpaye.total ?? 0),
  '▫ ?? 0 : si null (aucune facture impayée), on retourne 0
24  });
25  });
```



```

27 router.get("/dashboard/consommations-mensuelles", async (req, res) => {
    '▫ GET /api/dashboard/consommations-mensuelles — graphique barres
28     const MOIS_LABELS = ["Jan", "Fév", "Mar", "Avr", "Mai", "Jun",
    '▫ Tableau des noms de mois en français pour l'affichage
29     "Jul", "Aoû", "Sep", "Oct", "Nov", "Déc"];
30     const rows = await db.select({
    '▫ Requête d'agrégation groupée par mois et année
31     mois: consommationsTable.mois, annee: consommationsTable.annee,
    '▫ Colonnes de groupage
32     totalM3: sum(consommationsTable.m3Consommes),
    '▫ SUM(m3_consommes) GROUP BY annee, mois
33     }).from(consommationsTable)
34     .groupBy(consommationsTable.annee, consommationsTable.mois)
    '▫ GROUP BY annee, mois
35     .orderBy(consommationsTable.annee, consommationsTable.mois)
    '▫ Tri chronologique
36     .limit(12);
    '▫ Maximum 12 mois (1 an de données)
37     return res.json(rows.map(r => ({
    '▫ Transformation pour le graphique Recharts
38     totalM3: Number(r.totalM3 ?? 0),
    '▫ Conversion string !> number
39     label: `${MOIS_LABELS[r.mois - 1]} ${r.annee}`,
    '▫ Étiquette : "Jan 2025", "Fév 2025", etc.
40     })));
41 });

```


4. FRONTEND REACT — APPLICATION

Le frontend est construit avec React (bibliothèque UI) et Vite (outil de build). Il communique avec le backend via des hooks React Query générés automatiquement par Orval depuis la spécification OpenAPI.

4.1 `src/App.tsx` — Point d'entrée et routage




```

1  import { Switch, Route, Router as WouterRouter } from "wouter";
    '▫ Wouter : routeur léger pour React (alternative à React Router)
2  import { QueryClient, QueryClientProvider } from "@tanstack/react-query";
    '▫ TanStack Query : gestion du cache API, refetch automatique
3  import { Toaster } from "@components/ui/toaster";
    '▫ Composant pour afficher les notifications (toasts)
4  import Layout from "@components/layout";
    '▫ Layout = sidebar + zone de contenu principale
5  import Home from "@pages/home";
    '▫ Page d'accueil (non protégée, visible sans layout)
6  import Dashboard from "@pages/dashboard";
    '▫ Tableau de bord avec statistiques et graphiques
7
8  const queryClient = new QueryClient({
    '▫ Création du client de cache global pour toute l'application
9    defaultOptions: { queries: { retry: 1, refetchOnWindowFocus: false } },
    '▫ 1 retry max, pas de rechargement au focus de fenêtre
10 });
11
12 function AppRoutes() {
    '▫ Composant qui définit toutes les routes de l'application
13   return (
14     <Switch>
    '▫ Switch : seule la première route correspondante est rendue
15     <Route path="/" component={Home} />
    '▫ Route "/" ! Page d'accueil (sans sidebar)
16     <Route path="/dashboard">
    '▫ Route "/dashboard" ! Tableau de bord avec sidebar
17       <Layout><Dashboard /></Layout>
    '▫ Layout enveloppe le contenu avec la sidebar
18     </Route>
19     <Route path="/clients/:id">
    '▫ Route dynamique : :id est remplacé par l'ID réel (ex: /clients/3)
20       <Layout><ClientDetail /></Layout>
    '▫ Fiche détaillée d'un abonné
21     </Route>
22     <Route path="/clients">
    '▫ Liste des abonnés
23     <Route path="/consommations">
    '▫ Relevés de consommation
24     <Route path="/factures">
    '▫ Gestion des factures
25     <Route path="/reclamations">
    '▫ Gestion des réclamations
26     <Route component={NotFound} />
    '▫ Fallback : toute URL non reconnue ! page 404
27   </Switch>
28   );
29 }
30
31 function App() {
    '▫ Composant racine : fournit le contexte à toute l'application
32   return (

```



```
<QueryClientProvider client={queryClient}>
```



▣ *Rend le cache API disponible partout dans l'arbre React*


```
<WouterRouter base={import.meta.env.BASE_URL.replace(/\$/, "")}>
```

▣ *Routeur avec le chemin de base (ex: "/" en dev)*


```
<AppRoutes />
```


</WouterRouter>


```
<Toaster />
```

▣ Rendu des notifications toast (en dehors du routeur)

</QueryClientProvider>


```
export default App;
```

▣ Export par défaut pour être utilisé dans main.tsx

4.2 src/pages/home.tsx — Page d'accueil

```
1  import { Link } from "wouter";  
    '▫ Link : composant de lien de Wouter (navigation sans rechargement de page)  
2  import { Droplet, Users, ... } from "lucide-react";  
    '▫ Icônes SVG de la bibliothèque Lucide React  
3  
4  export default function Home() {  
    '▫ Composant de page exporté par défaut  
5    return (  
6      <div style={{ background: "linear-gradient(135deg, #0f2942...)"} >>  
        '▫ Dégradé CSS de fond : bleu marine !' bleu profond !' bleu marine  
7  
8      { /* HEADER */}  
9      <header className="px-8 py-5 flex items-center justify-between">  
        '▫ En-tête : logo à gauche, bouton à droite (flexbox)  
10     <div className="flex items-center gap-3">  
        '▫ Groupe logo + nom de l'entreprise  
11     <Droplet className="h-5 w-5 text-sky-300" />  
        '▫ Icône goutte d'eau (symbole REGIDESO)  
12     <p className="text-white font-bold text-lg">REGIDESO</p>  
        '▫ Nom en blanc, gras  
13     </div>  
14     <Link href="/dashboard">  
        '▫ Le lien navigue vers /dashboard  
15     <button data-testid="btn-acces-systeme-header">  
        '▫ data-testid : attribut pour les tests automatisés  
16     Accéder au système  
        '▫ Texte du bouton  
17     </button>  
18     </Link>  
19     </header>
```



```

21      { /* CONTENU PRINCIPAL */ }
22      <main className="flex-1 flex flex-col items-center justify-center">
23          '▢ Main centré verticalement et horizontalement
24          <div className="bg-sky-400/10 border border-sky-400/20 ... rounded-full">
25              '▢ Badge "Plateforme officielle de gestion" : fond semi-transparent
26              Plateforme officielle de gestion
27          </div>
28          <h1 className="text-5xl font-extrabold text-white">
29              '▢ Titre principal : taille 5xl (48px), ultra-gras, blanc
30              Gestion des Abonnés
31          <span className="block text-sky-400">REGIDES0</span>
32          '▢ Deuxième ligne en bleu ciel (block = va à la ligne)
33          </h1>
34
35      { /* ENCART PROJET TUTORÉ */ }
36      <div className="bg-white/5 border border-white/10 rounded-xl px-6 py-4">
37          '▢ Carte semi-transparente : fond blanc à 5% d'opacité
38          <p className="text-sky-300 text-xs font-semibold uppercase">
39              '▢ Titre de la carte en bleu ciel, majuscules, très petit
40              Projet tutoré – Fin de cycle de Licence
41          </p>
42          <p>Ce système a été conçu et réalisé par
43              <span className="font-semibold text-white">Ngandu Ngandu Jean</span>,
44              '▢ Nom de l'auteur en blanc et gras
45              étudiant en <span>L3 Informatique de Gestion</span>
46              '▢ Filière en gras
47          </p>
48      </div>

```



```

42     {/* GRILLE DES 4 MODULES */}
43     <div className="grid grid-cols-2 md:grid-cols-4 gap-4">
44         '▫ Grille responsive : 2 colonnes sur mobile, 4 sur grand écran'
45         {[
46             { icon: Users, label: "Abonnés", desc: "..."},
47             '▫ Données du module Abonnés'
48             { icon: Droplet, label: "Consommations", desc: "..."},
49             '▫ Données du module Consommations'
50             { icon: FileText, label: "Factures", desc: "..."},
51             '▫ Données du module Factures'
52             { icon: AlertTriangle, label: "Réclamations", desc: "..."},
53             '▫ Données du module Réclamations'
54             ].map(({ icon: Icon, label, desc }) => (
55                 '▫ Pour chaque élément, on crée une carte'
56                 <div key={label} className="bg-white/5 rounded-xl p-5">
57                     '▫ key={label} : clé unique React pour la réconciliation du DOM'
58                     <Icon className="h-4 w-4 text-sky-400" />
59                     '▫ Icône du module en bleu ciel'
60                     <p className="text-white font-semibold">{label}</p>
61                     '▫ Nom du module (Abonnés, Factures...)'
62                     <p className="text-slate-400 text-xs">{desc}</p>
63                     '▫ Description courte en gris'
64                 </div>
65             )))
66     </div>
67
68     <footer className="flex flex-col md:flex-row justify-between gap-2">
69         '▫ Pied de page : copyright à gauche, auteur à droite'
70         <p>© {new Date().getFullYear()} REGIDESO...</p>
71         '▫ Année dynamique : calculée à l'exécution (2026)'
72         <p>Réalisé par Ngandu Ngandu Jean – L3 Informatique de Gestion</p>
73         '▫ Mention de l'auteur et de la filière'
74     </footer>
75 </div>
76 );
77 }

```


4.3 src/pages/dashboard.tsx — Tableau de bord

```
1  import { useGetDashboardStats, useGetConsommationsMensuelles,  
    '▫ Hooks React Query générés par Orval depuis la spec OpenAPI'  
2      useGetReclamationsParType } from "@workspace/api-client-react";  
    '▫ Ces hooks appellent automatiquement les endpoints backend'  
3  import { BarChart, Bar, PieChart, Pie, Cell, ... } from "recharts";  
    '▫ Composants graphiques de la bibliothèque Recharts'  
4  
5  const COLORS = ['hsl(var(--chart-1))', 'hsl(var(--chart-2))', ...];  
    '▫ Couleurs CSS custom properties pour le graphique camembert'  
6  
7  export default function Dashboard() {  
    '▫ Composant principal du tableau de bord'  
8      const { data: stats, isLoading: statsLoading } = useGetDashboardStats();  
    '▫ Hook : déclenche GET /api/dashboard/stats, retourne les données et l'état'  
9      const { data: consoMensuelle } = useGetConsommationsMensuelles();  
    '▫ Hook : données pour le graphique à barres (consommations par mois)'  
10     const { data: reclamations } = useGetReclamationsParType();  
    '▫ Hook : données pour le graphique en anneau (réclamations par type)'  
11  
12     const isLoading = statsLoading || consoLoading || reclLoading;  
    '▫ true si au moins une requête est en cours'  
13     if (isLoading || !stats) return <SkeletonUI />;  
    '▫ Pendant le chargement, on affiche des squelettes animés'
```



```

15     return (
16       <div className="space-y-6 animate-in fade-in duration-500">
17         '▫ Animation d'entrée en fondu sur 500ms'
18         <h1>Tableau de bord</h1>
19         { /* CARTES DE STATISTIQUES */ }
20         <div className="grid gap-4 md:grid-cols-2 lg:grid-cols-4">
21           '▫ Grille : 1 col mobile, 2 col tablette, 4 col bureau'
22           <Card>
23             '▫ Carte "Total Abonnés"'
24             <div className="text-2xl font-bold">
25               {stats.totalClients.toLocaleString()}
26             </div>
27             '▫ toLocaleString() : formate le nombre selon la locale (ex: 1 234)'
28             <p>{stats.clientsActifs.toLocaleString()} actifs</p>
29             '▫ Nombre d'actifs comme sous-titre de la carte'
30           </Card>
31
32           <Card> { /* Impayés */ }
33             '▫ Carte des montants impayés, affichée en rouge'
34             <div className="text-destructive">
35               '▫ text-destructive = rouge (défini dans le thème CSS)'
36               {stats.montantImpaye.toLocaleString()} FC
37             </div>
38             '▫ Montant total en Francs Congolais'
39           </Card>
40         </div>
41       </div>
42     )

```

```

35     { /* GRAPHIQUE À BARRES : CONSOMMATIONS MENSUELLES */ }
36     <ResponsiveContainer width="100%" height="100%">
37       '▫ ResponsiveContainer adapte le graphique à la taille du conteneur'
38       <BarChart data={consoMensuelle}>
39         '▫ Graphique à barres avec les données de consommation mensuelle'
40         <CartesianGrid strokeDasharray="3 3" vertical={false} />
41         '▫ Lignes de grille horizontales (3px plein, 3px vide)'
42         <XAxis dataKey="label" axisLine={false} tickLine={false} />
43         '▫ Axe X : étiquettes des mois (Jan 2025, Fév 2025...)'
44         <YAxis axisLine={false} tickLine={false} />
45         '▫ Axe Y : valeurs en m³'
46         <Tooltip cursor={{ fill: 'var(--muted)' }} />
47         '▫ Info-bulle au survol avec fond coloré'
48         <Bar dataKey="totalM3" fill="hsl(var(--primary))" radius={[4,4,0,0]} />
49         '▫ Barre avec couleur primaire du thème, coins arrondis en haut'
50       </BarChart>
51     </ResponsiveContainer>

```



```
46      { /* GRAPHIQUE EN ANNEAU : RÉCLAMATIONS PAR TYPE */ }
47      <PieChart>
    '▢ Graphique circulaire (camembert / anneau)
48      <Pie data={reclamations}
    '▢ Données : tableau [{type, count}]
49      innerRadius={80} outerRadius={120}
    '▢ innerRadius > 0 !' forme d'anneau (donut)
50      paddingAngle={2} dataKey="count" nameKey="type">
    '▢ Espacement entre segments, clé valeur et clé nom
51      {reclamations.map((entry, index) => (
    '▢ Une couleur différente par type de réclamation
52      <Cell key={...} fill={COLORS[index % COLORS.length]} />
    '▢ % COLORS.length : cycle si plus de couleurs que de types
53      )))}
54      </Pie>
55      <Legend verticalAlign="bottom" height={36} />
    '▢ Légende sous le graphique
56      </PieChart>
```


FIN DU DOCUMENT

Système de Gestion des Clients REGIDESO

Réalisé par
NGANDU NGANDU JEAN
L3 Informatique de Gestion
Projet tutoré — Fin de cycle de Licence

© 2026 REGIDESO — Tous droits réservés.

